

PROJECT REPORT

LIBRARY MANAGEMENT SYSTEM

Course	CSE2006 – Programming in Java
Programme	B.Tech
Submitted By	Ayush Shukla
Registration No.	25BAI10014
Faculty / Guide	Dheresh Soni
Institution	VIT BHOPAL
Academic Year	2026–27

Submitted as part of the course requirements for CSE2006 – Programming in Java.

1. Introduction

The Library Management System is a Java desktop application created to demonstrate basic object-oriented concepts in an interactive graphical user interface. The application offers Librarian and Member user roles and manages book, member, search, issue, return, and storage operations. It features Java Swing for the graphical interface, Java Collections for in-memory data structures, File I/O for storage, custom exceptions for control flow, and background auditing threads for concurrency.

The Library Management System is a practice project, which aims to demonstrate fundamental concepts of object-oriented programming. The project's codebase is written in Java.

2. Problem Statement

Keeping track of library records can become tedious, both for books and members. This includes adding and removing books, registering and viewing members, searching, issuing, and returning books, as well as viewing issued books.

The Library Management System aims to provide a simple Java Swing GUI application to manage books, members, and issued books with concurrency and persistence.

3. Functional Requirements

- User roles: Librarian, Member
- Add/Remove books for Librarian
- Add/Register, View members for Librarian
- Search books by ID/title/author/category for both roles
- View available, issued books and issue a book for Member
- View issued books and check-in (return) for Librarian
- Availability state transitions between AVAILABLE and ISSUED for issued books
- Books borrowing limit to 3 issued copies per Member
- Input validation and exception handling for duplicates, invalid logins, unavailable books, invalid check-ins, and missing records
- Books, members, and issued books are persisted in text files.

4. Non-functional Requirements

- Usability: Basic Java Swing forms, buttons, tables, badges, and alerts
- Performance: In-memory ArrayList for adequate performance for this scale project
- Reliability: Input validation and custom exceptions for reliability
- Persistence: File I/O for storage
- Maintainability: Separation of concerns between domain classes, library operations, and GUI
- Portability: No external dependencies apart from Java
- Consistency: Synchronized library operations for consistent state

5. System Architecture

The application follows a layered, object-oriented desktop architecture. The application uses Java Swing for the GUI (presentation layer) and delegates operations to the business layer (Library). The business layer manages books, members, and issued books with Java Collections and persists them with File I/O.

Presentation Layer	Java Swing screens: Welcome, Login, Librarian Dashboard, Member Dashboard
---------------------------	---

Application / Controller	LibraryManagementSystem handles navigation, forms, event listeners, table refreshes, and dialogs
Business Layer	Library and LibraryOperations manage book, member, issue, return, search, and validation logic
Domain Model	User, Librarian, Member, Book, IssueRecord, BookStatus
Persistence Layer	books.txt, members.txt, issues.txt using Java File I/O

6. Design Diagrams

6.1 Use Case Diagram

Librarian → Login, Add Book, Remove Book, Search Book, Register Member, View Members, View Issued Books, Return Book, Logout

Member → Login, Search/View Books, Issue Book, View My Issued Books, View Profile, Logout

6.2 Workflow Diagram

Start → Load saved data → If data unavailable, load sample data → Show Welcome Screen → Select role → Authenticate user → Open relevant dashboard → Perform operation → Validate request → Update collections and status → Save text files → Refresh UI → Logout/Exit

6.3 Sequence Diagram: Issue Book

Member → Swing UI: Select Issue Book
Swing UI → Library: issueBook(bookId, member)
Library → Book Collection: Find book and check status
Library → Member: Check borrowing limit
Library → IssueRecord: Create issue record
Library → Files: Save books, members and issues
Library → Audit Thread: Log issue event
Library → Swing UI: Refresh tables and display success message

6.4 Class / Component Diagram

User (abstract)
■ ■ ■ Librarian
■ ■ ■ Member (department, issuedBookIds, maxBooks)

Book (bookId, title, author, category, status)
IssueRecord (bookId, memberId, issueDate)
BookStatus { AVAILABLE, ISSUED }

LibraryOperations (interface)
■ ■ ■ Library implements LibraryOperations
■ ■ ■ ArrayList<Book>
■ ■ ■ ArrayList<Member>
■ ■ ■ ArrayList<IssueRecord>

LibraryManagementSystem → Swing UI + CardLayout + event handlers

6.5 ER / Storage Diagram

MEMBER (memberId, name, password, department) 1 ■ ■ ■ * ISSUE_RECORD (bookId, memberId, issueDate) * ■ ■ ■ 1 BOOK (bookId, title, author, category, status)

Physical persistence: members.txt ↔ Member records; books.txt ↔ Book records; issues.txt ↔ IssueRecord records.

7. Design Decisions & Rationale

- Java Swing is used because the project is a desktop application, and Swing provides the

necessary GUI controls.

- CardLayout is used to switch between the welcome, login, and dashboard screens while keeping the application window on top.
- An abstract User class is used to reduce code duplication and demonstrate abstraction, encapsulation, and inheritance concepts. The Librarian and Member classes extend the User class.
- An interface, LibraryOperations , is defined for the library operations to demonstrate interface implementation.
- An enum is used for book availability to restrict values to AVAILABLE and ISSUED .
- Custom exceptions demonstrate exception handling and prevent invalid state transitions for concurrency and other operations.
- File I/O is used to persist books, members, and issued books in text files. This approach keeps the project simple by avoiding database and JDBC dependencies.
- Synchronized methods and a background auditing thread demonstrate concurrency and multithreading concepts.

8. Implementation Details

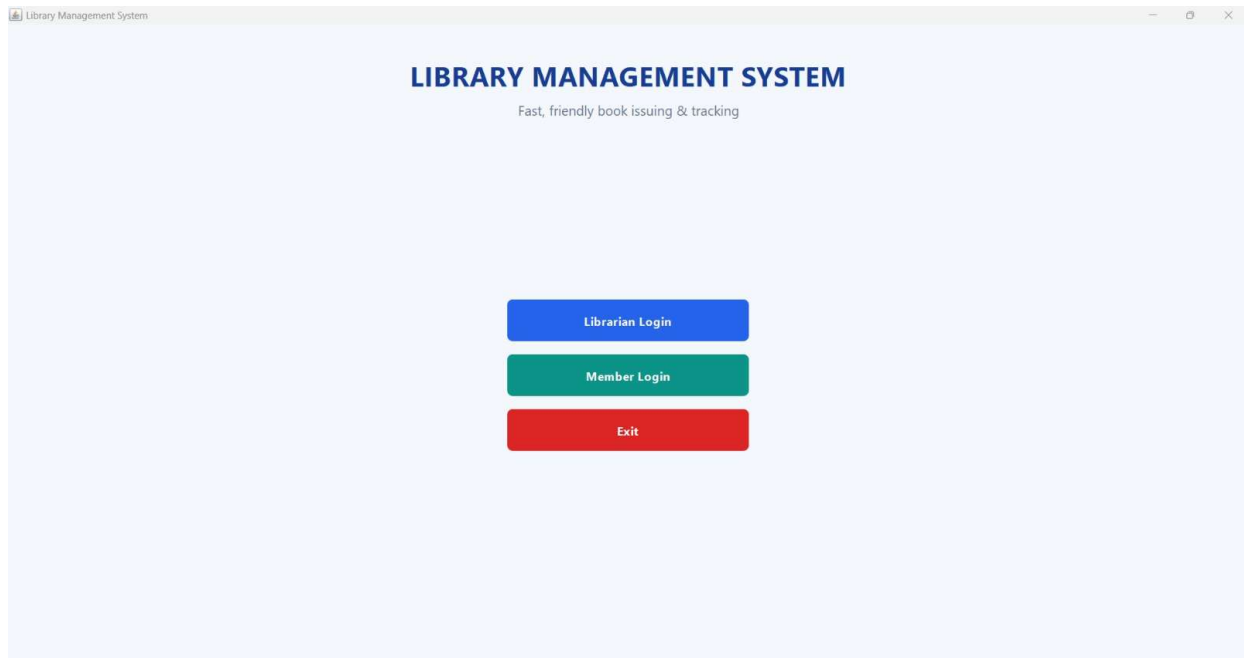
The project is a Java desktop application with GUI built with Java Swing. Books, members, and issued books are stored in ArrayList in-memory data structures. The primary domain classes are User , Librarian , Member , Book , IssueRecord , and Library . The LibraryManagementSystem class constructs the GUI and manages interactions with the domain classes.

The project implements object-oriented concepts, including abstraction, encapsulation, inheritance, polymorphism, interfaces, and method overriding. The Java Collections API is used for in-memory storage, and the File I/O API is used for persistence. Other features include GUI

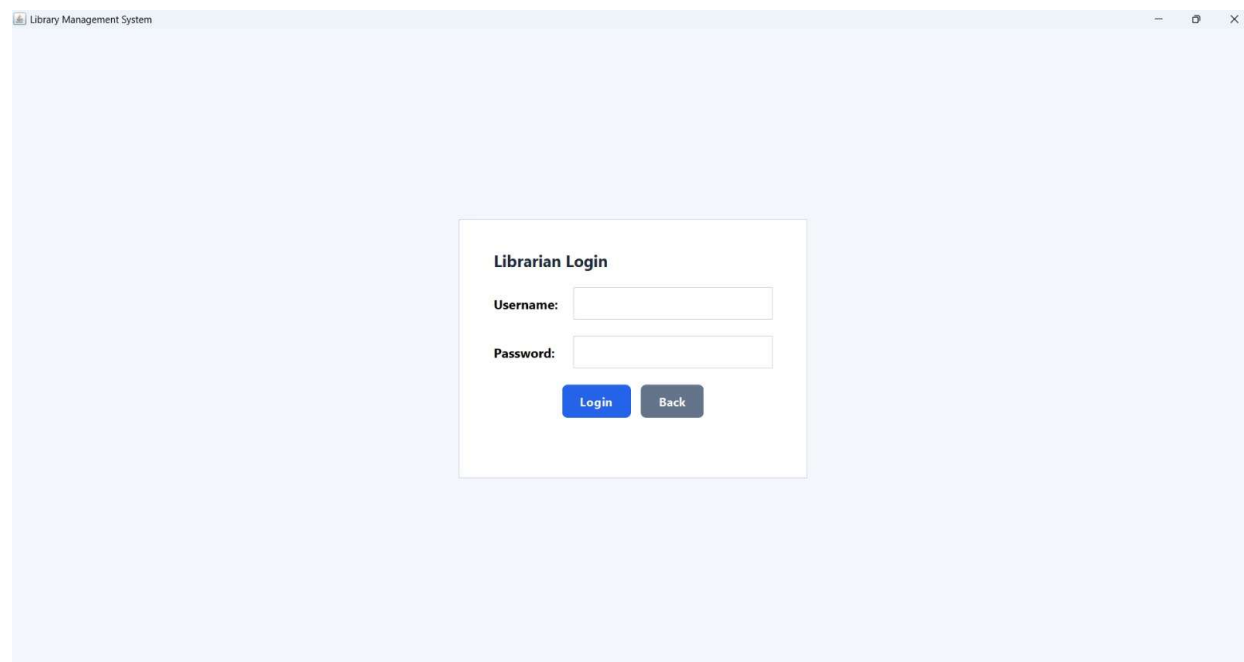
components such as JFrame , JPanel , CardLayout , JTable , JTabbedPane , JOptionPane , JButton , JTextField , and JPasswordField . The project also implements background auditing threads.

9. Screenshots / Results

(a) Welcome screen with Librarian login ,Member login and Exit



(b) Librarian login screen



(c) Librarian dashboard - All Books Tab

Library Management System

Welcome, Administrator (LIBRARIAN)

Logout

All BooksMembersIssued Books

B101	Java Programming	Herbert Schildt	CS	AVAILABLE
B102	Data Structures	Mark Allen Weiss	CS	AVAILABLE
B103	Operating System Concepts	Silberschatz	CS	AVAILABLE
B104	Computer Networks	Andrew Tanenbaum	CS	AVAILABLE
B105	Database System Concepts	Korth	CS	AVAILABLE

Add Book

Remove Book

Search:

Search

Show All

(d) Librarian Dashboard - Members Tab

Library Management System

Welcome, Administrator (LIBRARIAN)

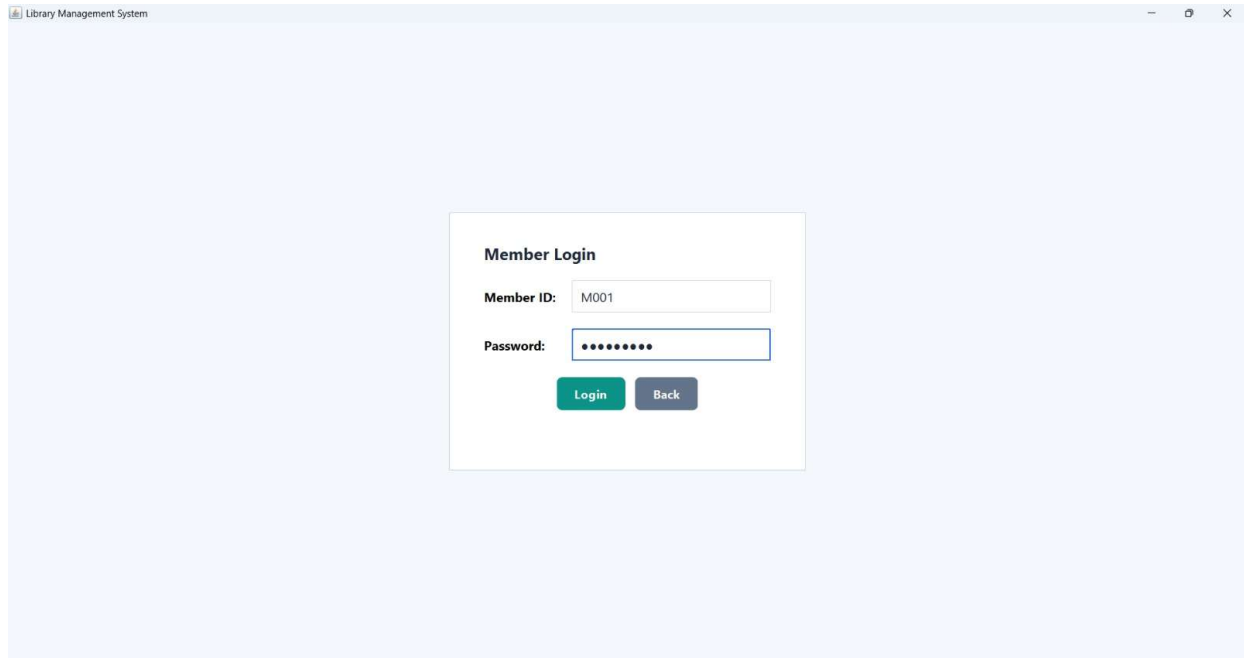
Logout

All BooksMembersIssued Books

M001	Aarav Sharma	CSE	0
M002	Priya Verma	IT	0

Register New Member

(e) Member Login Screen



The screenshot shows a web browser window titled "Library Management System". The main content area is light blue. In the center, there is a white rectangular box with a thin border. Inside this box, the text "Member Login" is displayed at the top. Below it, there are two input fields. The first is labeled "Member ID:" and contains the text "M001". The second is labeled "Password:" and contains a series of dots. Below the input fields, there are two buttons: a green "Login" button and a grey "Back" button.

Library Management System

Member Login

Member ID: M001

Password:

Login Back

(f) Member Dashboard - All books

Library Management System

Welcome, Aarav Sharma (M001) [Logout](#)

[All Books](#) [My Issued Books](#) [My Profile](#)

B101	Java Programming	Herbert Schildt	CS	AVAILABLE
B102	Data Structures	Mark Allen Weiss	CS	AVAILABLE
B103	Operating System Concepts	Silberschatz	CS	AVAILABLE
B104	Computer Networks	Andrew Tanenbaum	CS	AVAILABLE
B105	Database System Concepts	Korth	CS	AVAILABLE

[Issue Selected Book](#) Search: [Search](#) [Show All](#)

(g) Member Dashboard - My Profile

Library Management System

Welcome, Aarav Sharma (M001) [Logout](#)

[All Books](#) [My Issued Books](#) [My Profile](#)

My Profile

Member ID: M001

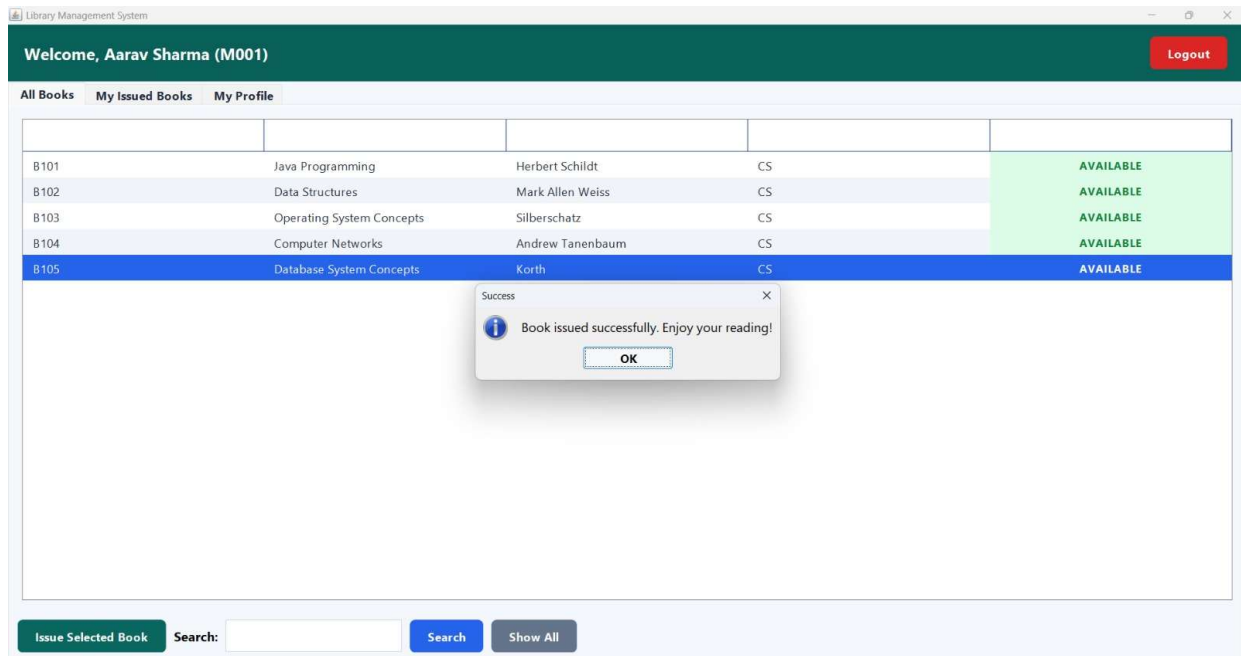
Name: Aarav Sharma

Department: CSE

Books Issued: 0

Max Allowed: 3

(h) Successful Book issue



10. Testing Approach

The project was tested using test cases that follow functional and boundary scenarios.

Login – Librarian	Valid credentials	Dashboard opens
Login – Member	Valid ID and password	Member dashboard opens
Invalid Login	Wrong password / unknown ID	Error message shown
Add Book	Unique book ID	Book added and saved
Duplicate Book	Existing book ID	DuplicateBookException handled
Remove Book	Available book	Book removed and saved
Remove Issued Book	Issued book ID	Operation rejected
Search	ID/title/author/category keyword	Matching records displayed
Issue Book	Available book, below limit	Status becomes ISSUED and record saved
Issue Unavailable Book	Already issued book	Operation rejected
Borrow Limit	Fourth issue attempt	BorrowLimitException handled
Return Book	Correct book/member relationship	Status becomes AVAILABLE and record removed
Persistence	Restart application	Saved records reloaded

11. Challenges Faced

- Maintaining book, member, and issue record consistency when issuing and returning books
- Avoiding duplicate records when adding books and registering members
- Persisting multiple data structures in text file format
- Creating the Librarian and Member dashboards with CardLayout and JTabbedPane
- Refreshing JTable models to reflect updates to the data structures
- Demonstrating multiple concepts, including OOP, exceptions, collections, File I/O, and multithreading in a single project

12. Learnings & Key Takeaways

- Learned how to create applications with modularity and extensibility in mind by following OOP principles.
- Applied object-oriented concepts to develop a working application, including abstraction, inheritance, polymorphism, encapsulation, interfaces, and enums.
- Gained insights into developing Java desktop applications with Swing, including GUI components, layout managers, and event-driven programming.
- Learned how to use exceptions to enforce validations and handle errors in Java applications.
- Applied Java Collections and File I/O to build an application with persistence.
- Learned about maintaining consistency when working with related classes and data structures in Java applications.
- Gained an introduction to multithreading and concurrency in Java applications.

13. Future Enhancements

- Replace text file storage with JDBC and a relational database
- Encrypt passwords with hashing algorithms
- Add book and member editing capabilities
- Add advanced book search capabilities
- Store and manage multiple copies of the same book
- Allow issuing books to members with due dates and calculate fines for overdue returns
- Generate reports and print issue records
- Add email notifications for due/overdue books
- Separate the code into multiple packages for production readiness

14. References

- Code: Library Management System Java source file attached for this report.

- Oracle Java Documentation – Java Platform and Standard Library documentation.
- Java Swing documentation.
- Course material for CSE2006 – Programming in Java.

Source Basis

Based on the submitted Java source code for the Library Management System. The source implements the Swing GUI, role-based login, book/member/issue management, custom exceptions and File I/O storage.